

ENSEMBLE METHODS

Bootstrap, Random Forest, AdaBoost, Gradient Boosting & XGBoost

TS. Cao Tiến Dũng



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

01

Bootstrap & Bagging

Lấy mẫu có hoàn lại, kết hợp song song, OOB evaluation

03

Boosting & AdaBoost

Weak learners, reweighting, Adaptive Boosting

05

XGBoost

Regularization, Taylor expansion, split gain

07

So sánh & Tổng kết

Bảng tổng hợp, decision flowchart, key takeaways

02

Random Forest

Feature randomness, max_features, Feature Importance

04

Gradient Boosting

Fit residuals, pseudo-residuals, learning rate

06

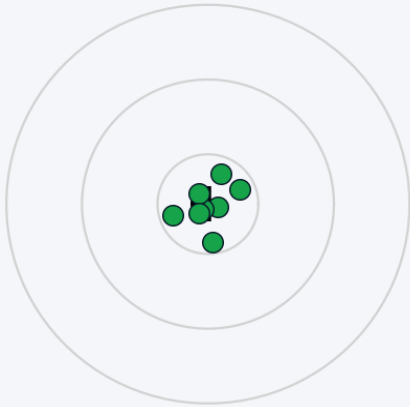
Bias–Variance & Thực hành

Phân rã lý thuyết, so sánh thực nghiệm, tuning

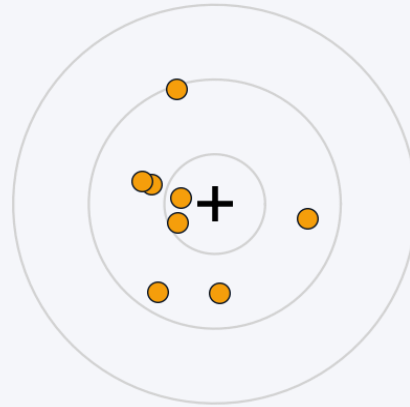
Vấn đề Single DT & Bias-Variance Tradeoff

2

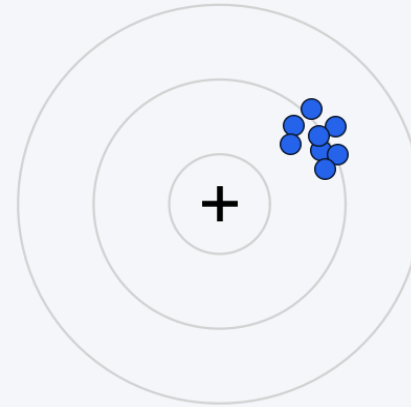
Low Bias
Low Variance



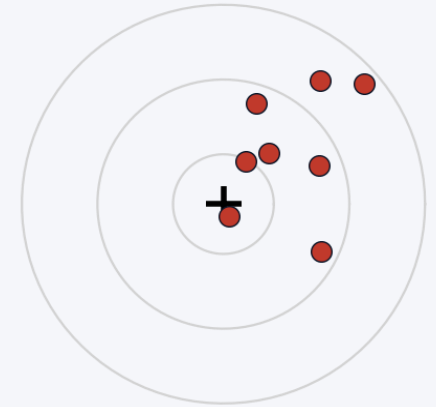
Low Bias
High Variance



High Bias
Low Variance



High Bias
High Variance



$$\text{Error}(x) = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Mục tiêu Ensemble: Giảm Variance (Bagging) hoặc giảm Bias (Boosting) — mà không tăng thành phần còn lại

Ý tưởng Ensemble & Phân loại

3

Bagging (Parallel)

Các model train song song, độc lập

→ Giảm Variance

Ví dụ: Bagging, Random Forest

Boosting (Sequential)

Các model train nối tiếp,
model sau sửa lỗi model trước

→ Giảm Bias

Ví dụ: AdaBoost, XGBoost

Stacking (Blending)

Kết hợp output nhiều model
khác loại bằng meta-learner

Ví dụ: Stacked Generalization

"Wisdom of the Crowd" — Tại sao Ensemble hoạt động?

Hội đồng bác sĩ: 7 bác sĩ hội chẩn → chính xác hơn 1 bác sĩ

Ban giám khảo: 7 người chấm → công bằng hơn 1 người

Nguyên tắc: Kết hợp nhiều weak learners → strong learner. Điều kiện: các model phải đa dạng (diverse) và tốt hơn random.

01

Bootstrap & Bagging

"Lấy mẫu có hoàn lại — sức mạnh từ sự ngẫu nhiên"



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Bootstrap Sampling là gì?

4

Định nghĩa

Lấy mẫu CÓ HOÀN LẠI (sampling with replacement) từ dataset gốc.

Mỗi bootstrap sample có cùng kích thước n với dataset gốc.

Một số mẫu xuất hiện nhiều lần, một số không xuất hiện.

Ví dụ: Dataset 5 mẫu [A, B, C, D, E]

	Vị trí 1	Vị trí 2	Vị trí 3	Vị trí 4	Vị trí 5
BS 1	A	C	C	D	A
BS 2	B	B	D	E	C
BS 3	A	D	D	D	E

Tính chất quan trọng

$P(\text{mẫu KHÔNG được chọn}) = (1 - 1/n)^n \approx e^{-1} \approx 0.368$

→ Mỗi bootstrap sample chứa ~63.2% mẫu gốc (unique), ~36.8% bị bỏ qua

36.8% bị bỏ qua → dùng làm "Out-of-Bag" (OOB) test set MIỄN PHÍ!



Minh hoạ Bootstrap — 10 mẫu

5

Mẫu gốc	BS 1	BS 2	BS 3	Xuất hiện (3 BS)
1	1x	—	2x	2/3
2	—	2x	—	1/3
3	2x	—	1x	2/3
4	—	1x	2x	2/3
5	1x	1x	—	2/3
6	—	2x	1x	2/3
7	2x	—	2x	2/3
8	1x	2x	—	2/3
9	2x	1x	—	2/3
10	1x	1x	2x	3/3

Quan sát

Mẫu 3, 7, 10: xuất hiện cả 3 BS

Mẫu 2: chỉ xuất hiện BS 2

Mỗi BS $\approx 63\%$ unique

Kết quả: Mỗi BS là phiên bản "hơi khác" của dataset gốc $\rightarrow$ diversity!

Tại sao Bootstrap?

1. Không cần thêm data mới
2. Mỗi BS $\rightarrow$ 1 model khác nhau
3. 36.8% OOB $\rightarrow$ validation miễn phí
4. Nền tảng cho Bagging & RF



Bagging = Bootstrap Aggregating (Breiman, 1996)

Input: Training set D (n mẫu, p features), số model B (thường 100–500)

For b = 1 to B:

1. Tạo bootstrap sample D_b (n mẫu, lấy có hoàn lại từ D)
2. Train Decision Tree T_b trên D_b (grow full, không pruning)

Prediction:

Classification: majority vote | **Regression:** trung bình

$$\text{Var}(\hat{f}_{bag}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

ρ = tương quan giữa các cây

σ^2 = variance mỗi cây

B = số cây

Khi $B \rightarrow \infty$: $\text{Var} \rightarrow \rho \sigma^2$

Giảm ρ = chìa khóa!

Bagging giảm Variance bằng trung bình hóa nhiều cây sâu (low bias, high variance). Nhưng ρ vẫn cao $\rightarrow$ Random Forest sẽ giải quyết.

	Tree 1	Tree 2	Tree 3	Tree 4	Tree 5	Voting
Sample A	High	Low	High	High	Low	→ High (3/5)
Sample B	Low	Low	Low	High	Low	→ Low (4/5)
Sample C	High	High	Low	Low	High	→ High (3/5)

Tại sao Bagging giảm Variance?

- 1. Mỗi cây có variance riêng
(vì train trên data khác nhau → predict khác nhau)
- 2. Trung bình hóa B predictions → triệt tiêu các sai số ngẫu nhiên
- 3. $\text{Var}(\text{trung bình}) = \sigma^2/B$ (nếu độc lập), thực tế $\approx \rho\sigma^2 + (1-\rho)\sigma^2/B$
- 4. **Bias không thay đổi — mỗi cây đã low bias (cây sâu)**

Vấn đề

Nếu 1 feature rất mạnh (dominant)

- TẤT CẢ cây đều chọn feature đó làm root split
- Các cây tương quan CAO (ρ lớn)
- Variance giảm ÍT

Ví dụ: Feature "Hút thuốc" quá mạnh

- 100 cây đều split "Hút thuốc?" ở root
- Các cây gần giống nhau
- Trung bình hóa KHÔNG giúp nhiều

Giải pháp → Random Forest

Tại mỗi split, chỉ xét m features ngẫu nhiên (thay vì tất cả p features)

- Feature dominant không phải lúc nào cũng được xét → cây KHÁC NHAU hơn
- ρ giảm → Variance giảm mạnh hơn

2 lớp Randomness:

- ① Data randomness (Bootstrap)
- ② Feature randomness (Random subset)

Bagging → tương quan cao do dominant features → Random Forest thêm feature randomness → giảm ρ → giảm variance mạnh hơn

Out-of-Bag (OOB) Evaluation

9

Cơ chế OOB

Với mỗi mẫu x_i :

1. Tìm tất cả cây mà x_i KHÔNG nằm trong bootstrap sample (~36.8% cây)
2. Dùng các cây đó predict cho x_i
3. Majority vote / average → OOB prediction
4. So sánh với nhãn thật → OOB score

Kết quả: Ước lượng test error

MÀ KHÔNG CẦN validation set riêng!

sklearn

```
rf = RandomForestClassifier(  
    n_estimators=100,  
    oob_score=True,  
    random_state=42  
)  
rf.fit(X_train, y_train)  
print(f'OOB: {rf.oob_score_:.3f}')
```

Tiêu chí	OOB	Cross-Validation
Chi phí	Miễn phí (built-in)	Tốn K lần train
Cần data riêng	Không	Không
Phù hợp	RF, Bagging	Mọi model
Khi nào dùng	Nhanh, baseline	Chính xác hơn



Công thức Variance của Bagging

$$\text{Var}(\hat{f}_{\text{bag}}) = \rho \sigma^2 + \frac{1-\rho}{B} \sigma^2$$

ρ = tương quan trung bình giữa các cây | σ^2 = variance mỗi cây | B = số cây

Trường hợp	ρ	$\text{Var}(f_{\text{bag}})$	Ý nghĩa
Lý tưởng (độc lập)	0	σ^2/B	Giảm tuyến tính theo B
Thực tế (tương quan)	0.3–0.7	$\rho\sigma^2 + (1-\rho)\sigma^2/B$	Có giới hạn dưới = $\rho\sigma^2$
Giống hệt ($\rho=1$)	1.0	σ^2	Không giảm gì!

Khi $B \rightarrow \infty$: $\text{Var} \rightarrow \rho\sigma^2$. Muốn giảm thêm $\rightarrow$ phải giảm ρ (tương quan). Đây là motivation cho Random Forest!

02

Random Forest

"Bagging + Feature Randomness → giảm tương quan → mạnh hơn"



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

Từ Bagging đến Random Forest

10

Tiêu chí	Bagging	Random Forest
Base learner	Decision Tree	Decision Tree
Bootstrap sampling	Có	Có
Feature tại mỗi split	Tất cả p features	m features ngẫu nhiên ★
Tương quan giữa cây	Cao	Thấp hơn ★
Variance reduction	Tốt	Tốt hơn ★
Bias	Thấp	Hơi cao hơn (trade-off)

Chọn m (max_features)

Classification: $m = \sqrt{p}$ (mặc định)

Regression: $m = p/3$ (mặc định)

Extra Trees: $m = 1$

= Bagging: $m = p$

m nhỏ $\rightarrow$ đa dạng, p thấp

m lớn $\rightarrow$ mỗi cây mạnh, p cao

$\rightarrow m$ là hyperparameter quan trọng!

RF = Bagging + Feature Randomness $\rightarrow$ giảm $p \rightarrow$ giảm variance mạnh hơn. Đây là "go-to baseline" cho tabular data.



Algorithm: Random Forest

Input: Training set D , số cây B , số features m

For $b = 1$ to B :

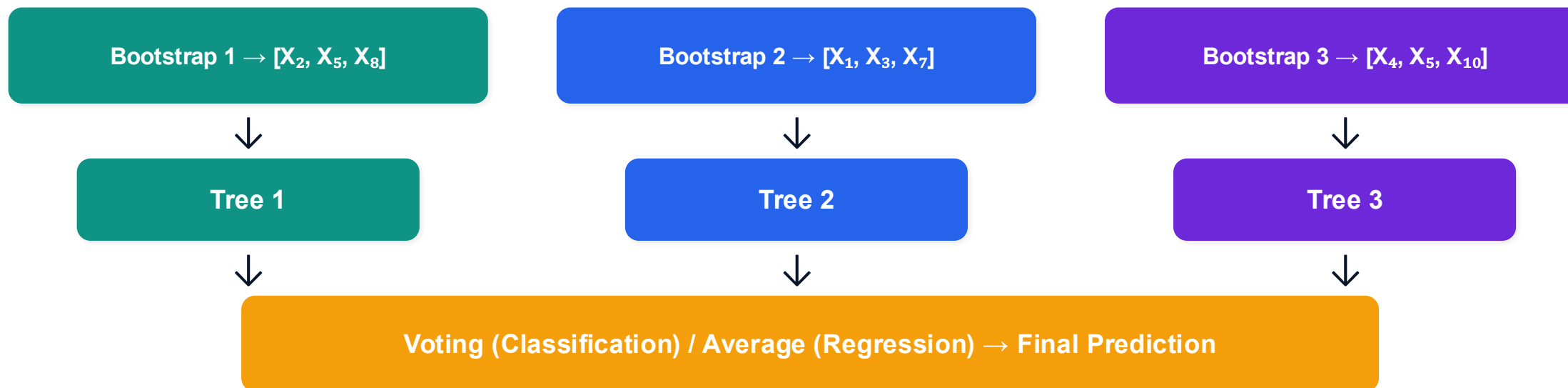
1. Tạo bootstrap sample D_β từ D
2. Train Decision Tree T_β trên D_β :
Tại mỗi node cần split:
 - **Chọn ngẫu nhiên m features từ p features**
 - **Tìm best split CHỈ trong m features đó**
 - Split node. Grow cây đến max (không pruning)

Prediction: Classification: $\hat{y} = \text{majority_vote}(T_1..T_\beta)$ | Regression: $\hat{y} = \text{mean}(T_1..T_\beta)$

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=200,    # Số cây
    max_features='sqrt', # m =  $\sqrt{p}$  (classification)
    max_depth=None,     # Grow full
    oob_score=True,
    n_jobs=-1,          # Parallel
    random_state=42
)
rf.fit(X_train, y_train)
print(f'Test: {rf.score(X_test, y_test):.3f} OOB: {rf.oob_score_: .3f}')
```

Training Data (p features) $\rightarrow$ B cây, mỗi cây dùng bootstrap + m features random



Mỗi cây 'nhìn' data KHÁC (bootstrap) VÀ features KHÁC (random subset) $\rightarrow$ CÂY ĐA DẠNG $\rightarrow$ p thấp $\rightarrow$ Variance giảm mạnh

So sánh: Bagging vs Random Forest

13

Tiêu chí	Bagging	Random Forest
Base learner	Decision Tree (full)	Decision Tree (full)
Bootstrap sampling	Có	Có
Feature tại mỗi split	Tất cả p features	m features ngẫu nhiên
Tương quan giữa cây	Cao (dominant feature)	Thấp (random features)
Variance reduction	Tốt (giới hạn bởi $p\sigma^2$)	Tốt hơn (p nhỏ hơn)
Bias	Thấp	Hơi cao hơn (trade-off khi m nhỏ)
Feature Importance	MDI (thiên vị)	MDI + Permutation
Hyperparameters	n_estimators	n_estimators, max_features

Random Forest = Bagging + Feature Randomness → giảm tương quan → variance giảm mạnh hơn. Công thức: Var giảm khi p giảm!



MDI (Gini Importance)

Mean Decrease in Impurity

Cách tính:

- Mỗi lần feature X_j split $\rightarrow$ Gini giảm ΔG
- Importance = trung bình ΔG trên tất cả cây

Có sẵn: `model.feature_importances_`

- ✓ Nhanh, built-in
- ⚠ Thiên vị features high cardinality

Permutation Importance

Cách tính:

1. Tính accuracy gốc trên test set
 2. Shuffle feature $X_j \rightarrow$ accuracy mới
 3. Mức giảm = importance
 4. Lặp lại `n_repeats` lần
- ✓ Không thiên vị, đo trên test set
 - ⚠ Chậm hơn, bị ảnh hưởng bởi correlation

```
from sklearn.inspection import permutation_importance  
result = permutation_importance(rf, X_test, y_test, n_repeats=10)
```

Nên dùng CẢ HAI và so sánh. Khi nghi ngờ $\rightarrow$ ưu tiên Permutation Importance.

Random Forest — Ưu điểm, Hạn chế & Khi nào dùng?

13

Ưu điểm

- Hiệu suất cao, ít tuning
- Không cần scaling
- Numerical & categorical
- Robust với outliers
- Feature importance sẵn
- OOB miễn phí
- Dễ parallel
- Khó overfit khi tăng B

Hạn chế

- Chậm hơn single tree
- Black-box (kém interpretable)
- Tốn memory (B cây)
- Thường kém hơn XGBoost
- Extrapolation kém
- Predict chậm khi deploy

Khi nào dùng?

- ✓ Baseline nhanh, mạnh
- ✓ Nhiều features, cần importance
- ✓ Data có noise, outliers
- ✓ Dataset nhỏ, dùng OOB
- ⚠ Accuracy tối đa → XGBoost
- ⚠ Interpretability → single tree
- ⚠ Real-time → model nhỏ hơn

Random Forest = "go-to baseline" cho tabular data. Nếu cần accuracy tối đa → chuyển sang Boosting (AdaBoost, GB, XGBoost).

03

Boosting & AdaBoost

"Kết hợp nhiều weak learners → strong learner"



FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Câu hỏi nền tảng (Kearns & Valiant, 1988)

"Liệu một tập hợp các weak learners có thể tạo thành strong learner?"

→ CÓ! (Schapire, 1990)

Weak Learner = model chỉ cần accuracy > 50%

Ví dụ: Decision Stump (cây depth=1)

Thực giác: Học sinh làm bài

Lần 1: Làm 100 câu, sai 10 câu

Lần 2: Ôn 10 câu sai → sửa 7 câu

Lần 3: Ôn 3 câu còn sai → sửa hết

→ Master toàn bộ!

Boosting: Model sau tập trung

sửa lỗi model trước

Tiêu chí	Bagging	Boosting
Chiến lược	Parallel, độc lập	Sequential, phụ thuộc
Mục tiêu	Giảm Variance	Giảm Bias
Base learner	Cây sâu (low bias)	Cây nông / stumps
Trọng số models	Đều nhau	Khác nhau (theo accuracy)
Overfit	Ít overfit	Có thể overfit nếu quá nhiều iter

AdaBoost — Tổng quan (Freund & Schapire, 1996)

15

Ý tưởng: Tăng trọng số sample bị phân loại SAI → model tiếp theo "chú ý" vào sample khó

- Base learner mặc định: Decision Stump (cây depth=1)
- Mỗi stump có trọng số α_t khác nhau (model tốt → α lớn)
- **Kết quả cuối: $H(x) = \text{sign}(\sum \alpha_t \cdot h_t(x))$**

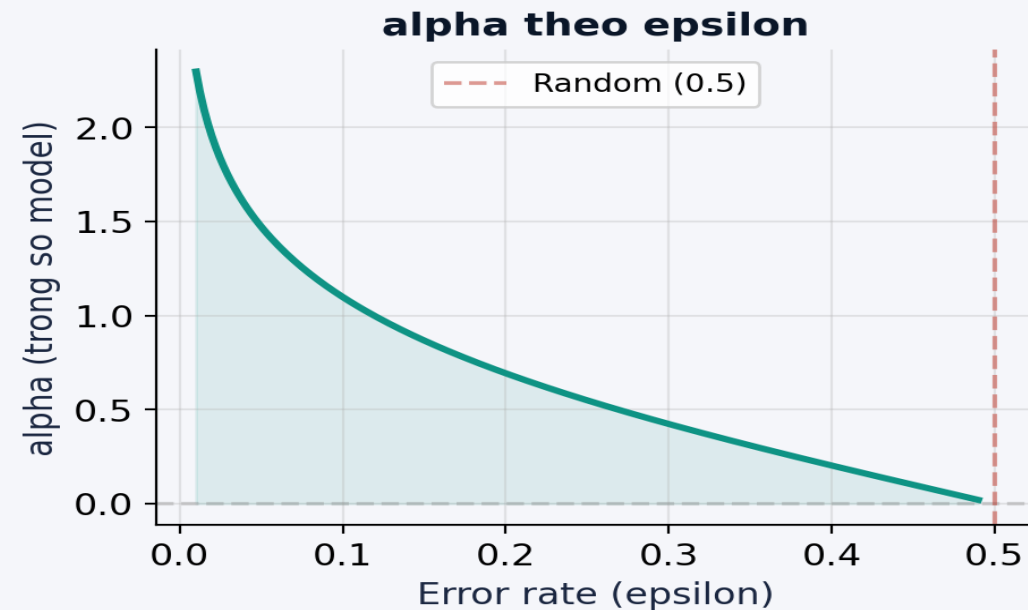
$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \varepsilon_t}{\varepsilon_t} \right)$$

Phân tích α_t :

$\varepsilon \rightarrow 0$ (perfect): $\alpha \rightarrow \infty$ (tin tưởng tuyệt đối)

$\varepsilon = 0.5$ (random): $\alpha = 0$ (bỏ qua)

$\varepsilon > 0.5$ (tệ hơn random): $\alpha < 0$ (đảo ngược)



Thuật toán AdaBoost — 4 Bước

16

1 Khởi tạo trọng số

$w_i = 1/n$ cho mỗi sample.
Tất cả sample được "chú ý" như nhau.

2 Train weak learner h_t

Train decision stump với trọng số w_i .
Tính weighted error: $\epsilon_t = \sum w_i \cdot I[y_i \neq h_t(x_i)]$

3 Tính trọng số model α_t

$\alpha_t = \frac{1}{2} \ln((1-\epsilon_t)/\epsilon_t)$
Model tốt (ϵ thấp) $\rightarrow \alpha$ lớn $\rightarrow$ ảnh hưởng nhiều hơn

4 Cập nhật trọng số sample

Sai: $w_i \times \exp(+\alpha_t) \rightarrow$ tăng mạnh
Đúng: $w_i \times \exp(-\alpha_t) \rightarrow$ giảm
Normalize để $\sum w_i = 1$

Lặp lại bước 2–4 cho T vòng $\rightarrow$ kết hợp:

$$H(x) = \text{sign}\left(\sum_{t=1}^T \alpha_t h_t(x)\right)$$

$$w_i \leftarrow w_i \cdot \exp(\alpha_t \cdot I[y_i \neq h_t(x_i)])$$

AdaBoost — Minh hoạ 3 Rounds

17

Round	Stump	Error ϵ_t	α_t	Reweight	Kết quả
1	$X_1 > 3?$	0.30	0.42	Sai: $w \times 1.52$ Đúng: $w \times 0.66$	3/10 sai
2	$X_2 > 5?$	0.21	0.66	Sai: $w \times 1.93$ Đúng: $w \times 0.52$	Tập trung vào 3 sai
3	$X_1 > 7?$	0.14	0.90	—	Hầu hết đúng

Combined: $H(x) = \text{sign}(0.42 \cdot h_1(x) + 0.66 \cdot h_2(x) + 0.90 \cdot h_3(x))$

Model 3 ($\alpha=0.90$) ảnh hưởng nhiều nhất vì error thấp nhất. Mỗi stump đơn lẻ yếu, nhưng kết hợp $\rightarrow$ strong learner!

- ✓ Tự chọn α , không cần tune learning rate
- ✓ Ít hyperparameter, dễ implement

- ⚠ Nhạy với noise & outliers (weight tăng mãi)
- ⚠ Chỉ exponential loss $\rightarrow$ hạn chế



Ưu điểm

- Ít hyperparameter (chỉ T và base learner)
- Tự chọn α — không cần tune learning rate
- Dễ implement, có lý thuyết vững chắc
- Hoạt động với BẤT KỲ weak learner
- Không dễ overfit (nếu data sạch)
- **Gödel Prize 2003 — nền tảng lý thuyết**

Hạn chế

- Nhạy cảm với noise & outliers
→ Sample noise bị weight tăng mãi
- Chỉ exponential loss → không linh hoạt
- Chậm hơn RF (sequential, không parallel)
- Thường kém XGBoost trên tabular data
- Base learner yếu → cần nhiều rounds

```
from sklearn.ensemble import AdaBoostClassifier
ada = AdaBoostClassifier(
    estimator=DecisionTreeClassifier(max_depth=1), # Stump
    n_estimators=100,
    learning_rate=0.1, # Shrinkage (sklearn thêm)
    algorithm='SAMME.R', # Real AdaBoost (probability-based)
    random_state=42
)
ada.fit(X_train, y_train)
print(f'Train: {ada.score(X_train, y_train):.3f}')
print(f'Test: {ada.score(X_test, y_test):.3f}')
```

04

Gradient Boosting

"Fit residuals — sửa lỗi bằng gradient descent trong không gian hàm"



Hạn chế của AdaBoost

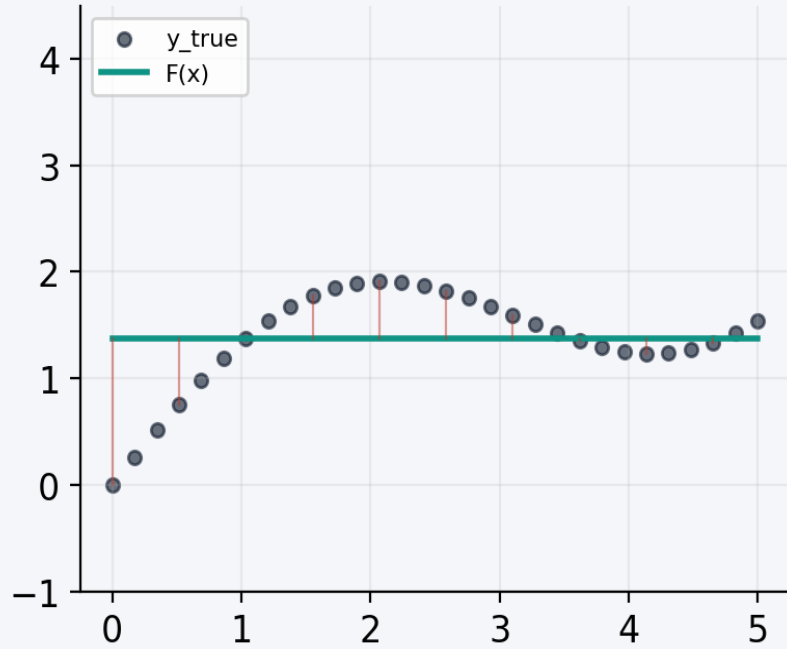
1. Chỉ tối ưu exponential loss
→ Nhạy với noise, outliers
2. Chỉ reweight sample
→ Không linh hoạt
3. Khó mở rộng sang regression
hoặc loss function khác
→ Cần framework TỔNG QUÁT hơn

Gradient Boosting (Friedman, 2001)

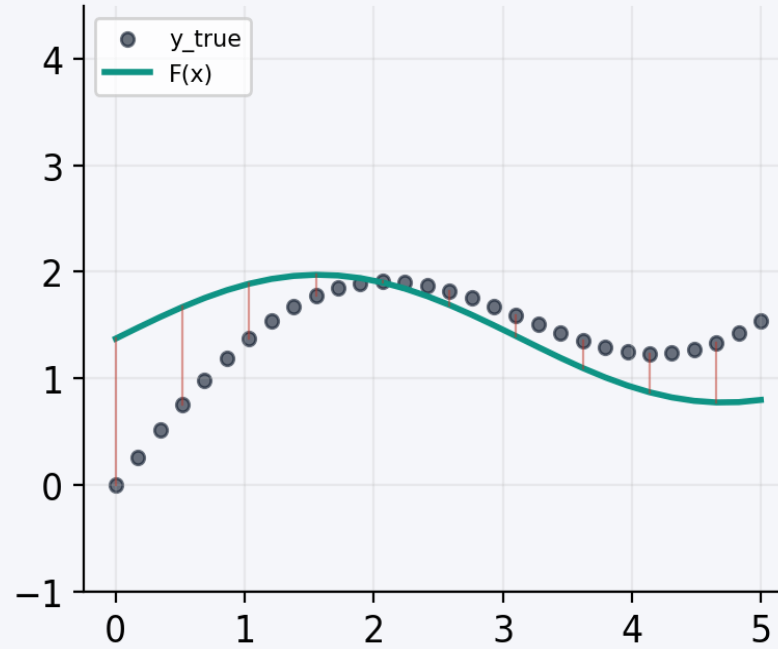
Ý tưởng: Thay vì reweight sample, fit model mới vào RESIDUALS (phần sai số) của model trước
→ Hoạt động với BẤT KỲ loss function nào có đạo hàm (MSE, MAE, Log Loss...)
→ Mỗi model mới = 1 bước gradient descent trong không gian hàm

AdaBoost: reweight samples (exponential loss only) → GB: fit residuals (ANY differentiable loss)

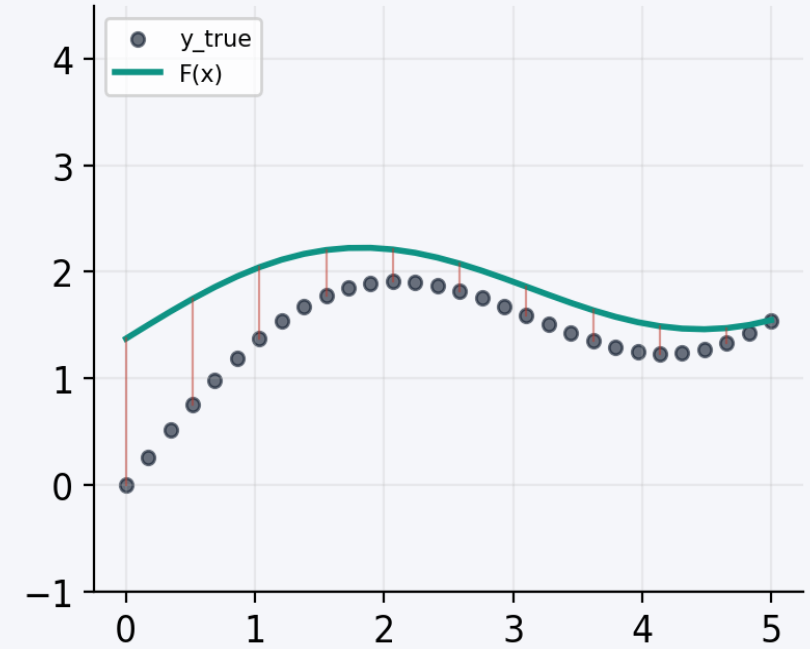
Round 1: $F_0 = \text{mean}(y)$



Round 2: Fit residual r_1



Round 3: Fit residual r_2



Round 1: $F_0 = \text{mean}(y) \rightarrow$ tính residual $r_1 = y - F_0$

Round 2: Fit tree h_1 vào $r_1 \rightarrow F_1 = F_0 + \eta \cdot h_1 \rightarrow r_2 = y - F_1$

Round 3: Fit tree h_2 vào $r_2 \rightarrow F_2 = F_1 + \eta \cdot h_2 \rightarrow$ residual ngày càng nhỏ!

Algorithm

- 1. Khởi tạo: $F_0(x) = \operatorname{argmin}_c \sum L(y_i, c)$ (vd: $F_0 = \operatorname{mean}(y)$ cho MSE)
- 2. For $m = 1$ to M :
 - a. Tính pseudo-residuals: $r_{im} = -\partial L(y_i, F(x_i)) / \partial F(x_i)$
 - b. Fit regression tree h_m vào r_{im}
 - c. Cập nhật: $F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$
- 3. Output: $F_m(x) = F_0 + \eta \cdot h_1 + \eta \cdot h_2 + \dots + \eta \cdot h_m$

$$r_{im} = -\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)}$$

$$F_m(x) = F_{m-1}(x) + \eta \cdot h_m(x)$$

Loss	$L(y, F)$	Pseudo-residual r_i	Dùng cho
MSE	$\frac{1}{2}(y-F)^2$	$y - F$ (residual thường)	Regression
MAE	$ y-F $	$\operatorname{sign}(y - F)$	Regression robust
Log Loss	$-y \cdot \log(p) - (1-y) \cdot \log(1-p)$	$y - p$ (probability)	Classification

Mẫu	y (thật)	$F_0=3.0$	$r_1=y-F_0$	$h_1(x)$	$F_1=F_0+0.1 \cdot h_1$	r_2	$h_2(x)$	F_2
x_1	2.0	3.0	-1.0	-0.95	2.905	-0.905	-0.88	2.817
x_2	3.5	3.0	+0.5	+0.48	3.048	+0.452	+0.44	3.092
x_3	4.0	3.0	+1.0	+0.95	3.095	+0.905	+0.88	3.183
x_4	2.5	3.0	-0.5	-0.48	2.952	-0.452	-0.44	2.908

$\eta = 0.1$ (learning rate)

$F_0 = \text{mean}(y) = 3.0 \rightarrow r_1 = y - 3.0 \rightarrow \text{Fit tree } h_1 \text{ vào } r_1 \rightarrow F_1 = F_0 + 0.1 \cdot h_1 \rightarrow r_2 = y - F_1 \rightarrow \dots \text{ lặp lại}$

Residuals ngày càng NHỎ qua mỗi round $\rightarrow$ mô hình ngày càng chính xác!

Tại sao gọi "Gradient"? Pseudo-residual $r_i = -\partial L / \partial F$ chính là negative gradient $\rightarrow$ mỗi cây = 1 bước gradient descent trong function space

Tại sao gọi "Gradient" Boosting?

22

Gradient Descent thông thường

Tối ưu THAM SỐ θ :

$$\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

Cập nhật tham số theo hướng

negative gradient

Ví dụ: Linear Regression

$$\theta = [w_1, w_2, \dots, w_p, b]$$

Cập nhật từng w_i

Gradient Boosting

Tối ưu HÀM SỐ $F(x)$:

$$F(x) \leftarrow F(x) - \eta \cdot \partial L / \partial F(x)$$

Cập nhật hàm F theo hướng

negative gradient

Nhưng $F(x)$ là hàm, không phải số!

→ Dùng tree h_m xấp xỉ $-\partial L / \partial F$

$$\rightarrow F_m = F_{m-1} + \eta \cdot h_m$$

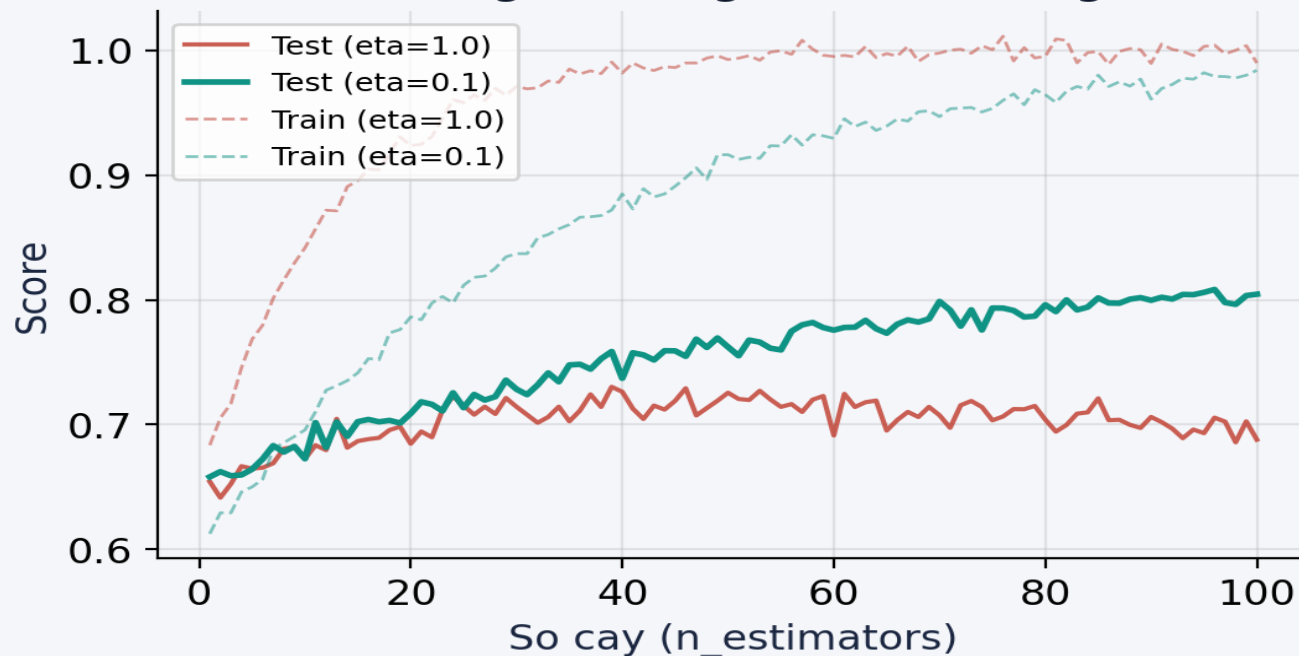
= Gradient Descent trong

FUNCTION SPACE!

GB = Gradient Descent trong function space. Mỗi cây mới = 1 bước gradient. Pseudo-residual = negative gradient của loss.



Hieu ung Learning Rate (Shrinkage)



Shrinkage (Learning Rate η)

$$F_m = F_{m-1} + \eta \cdot h_m(x)$$

η nhỏ (0.01–0.1):

- Mỗi cây đóng góp ít
- Cần nhiều cây hơn
- Nhưng tổng quát hoá TỐT hơn

η lớn (0.3–1.0):

- Hội tụ nhanh
- Dễ overfit

Trade-off: η nhỏ + B lớn = tốt nhất
(nhưng chậm hơn)

Kỹ thuật	Mô tả	Tác dụng
Shrinkage	$\eta \in [0.01, 0.3]$	Giảm overfit, cần nhiều cây
Subsampling	subsample $\in [0.5, 0.8]$	Stochastic GB (như RF)
max_depth	Thường 3–6	Giới hạn interaction depth
min_samples_leaf	≥ 5 –20	Tránh lá quá nhỏ
n_estimators	100–1000	Dùng early stopping

05

XGBoost

"eXtreme Gradient Boosting — Nhanh, chính xác, chiến thắng Kaggle"



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

Tại sao XGBoost thống trị?

- 17/29 winning solutions trên Kaggle (2015)
- Nhanh hơn GB truyền thống 10x+
- Regularization built-in
- Xử lý missing values tự động
- Parallel tree construction
- Cache-aware, out-of-core computing

3 cải tiến chính so với GB:

- ① **Regularized objective**
- ② **Newton's method (bậc 2)**
- ③ **Engineering optimizations**

```
import xgboost as xgb

model = xgb.XGBClassifier(
    n_estimators=200,
    max_depth=5,
    learning_rate=0.1,
    subsample=0.8,
    colsample_bytree=0.8,
    reg_lambda=1.0,    # L2 regularization
    reg_alpha=0.0,     # L1 regularization
    early_stopping_rounds=20,
    eval_metric='logloss',
    random_state=42
)
model.fit(X_train, y_train,
          eval_set=[(X_val, y_val)],
          verbose=False)
```

Regularized Objective — Điểm khác biệt với GB truyền thống

$$\text{Obj} = \sum_{i=1}^n L(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k)$$

*Loss function
(fit data)*

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

*Regularization
(kiểm soát complexity)*

Taylor Expansion (Bậc 2)

XGBoost xấp xỉ loss bằng Taylor bậc 2:

$$L(y_i, \hat{y}_i + f_t) \approx L(y_i, \hat{y}_i) + g_i f_t + \frac{1}{2} h_i f_t^2$$

$g_i = \partial L / \partial \hat{y}$ (gradient — bậc 1)

$h_i = \partial^2 L / \partial \hat{y}^2$ (hessian — bậc 2)

→ GB chỉ dùng bậc 1 (gradient)

→ XGBoost dùng cả bậc 2 → hội tụ nhanh hơn

Optimal Split Gain

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

GL, GR = tổng gradient trái/phải

HL, HR = tổng hessian trái/phải

λ = L2 regularization

γ = minimum loss reduction

Gain > 0 → split có lợi

Gain < 0 → KHÔNG split (pruning)

Xấp xỉ Taylor bậc 2 — Chìa khóa tốc độ XGBoost

GB truyền thống: chỉ dùng gradient bậc 1 ($g_i = \partial L / \partial \hat{y}$) → hội tụ chậm

XGBoost: dùng cả gradient (g_i) VÀ hessian ($h_i = \partial^2 L / \partial \hat{y}^2$) → Newton's method → hội tụ nhanh hơn

$$L(y_i, \hat{y}_i + f_t) \approx L(y_i, \hat{y}_i) + g_i \cdot f_t + \frac{1}{2} \cdot h_i \cdot f_t^2$$

Loss function	g_i (gradient)	h_i (hessian)
MSE: $\frac{1}{2}(y - \hat{y})^2$	$\hat{y} - y$	1
Log Loss	$p - y$	$p(1 - p)$
Huber loss	clipped $(\hat{y} - y)$	1 hoặc 0

Tại sao Hessian giúp?

Curvature information cho biết

"bước đi bao xa" là tối ưu.

→ Ít iterations hơn

→ Mỗi split chính xác hơn

Split Gain — Quyết định có nên split hay không

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} \right] - \gamma$$

G = tổng gradient | H = tổng hessian | λ = L2 reg | γ = min loss reduction

Thành phần	Ý nghĩa	Tác dụng
$G^2L / (HL+\lambda)$	Score nhánh trái	Đo chất lượng split trái
$G^2R / (HR+\lambda)$	Score nhánh phải	Đo chất lượng split phải
$(GL+GR)^2 / (HL+HR+\lambda)$	Score KHÔNG split	Benchmark: giữ nguyên
$-\gamma$	Penalty cho thêm lá	Pruning tự động!

Gain > 0 → split có lợi (giảm loss > cost thêm lá). Gain ≤ 0 → KHÔNG split → post-pruning tự động. GB truyền thống không có γ!

So sánh: GB truyền thống vs XGBoost

27

Tiêu chí	Gradient Boosting	XGBoost
Objective	Loss only	Loss + Regularization $\Omega(f)$
Optimization	Gradient bậc 1	Newton (gradient + hessian bậc 2)
Pruning	Pre-pruning (max_depth)	Post-pruning (Gain < 0, γ)
Missing values	Cần impute trước	Tự động (default direction)
Parallelism	Không (sequential)	Column-parallel split finding
Column subsampling	Chỉ row (subsample)	Row + Column (colsample_bytree)
Cache	Không tối ưu	Cache-aware, out-of-core
Tốc độ	Chậm	Nhanh 10x+
Accuracy	Cao	Rất cao

XGBoost = GB + Regularization + Newton's method + Engineering → chuẩn vàng cho tabular data competitions



**FPT SCHOOL OF BUSINESS
& TECHNOLOGY**

Tham số	Mô tả	Range	Khuyến nghị
<code>n_estimators</code>	Số cây (rounds)	50–1000	Dùng <code>early_stopping</code>
<code>learning_rate</code> (η)	Shrinkage	0.01–0.3	0.05–0.1 + <code>early stop</code>
<code>max_depth</code>	Độ sâu mỗi cây	3–10	3–6 (tabular)
<code>subsample</code>	% rows mỗi tree	0.5–1.0	0.7–0.8
<code>colsample_bytree</code>	% features mỗi tree	0.5–1.0	0.7–0.8
<code>reg_lambda</code> (λ)	L2 regularization	0–10	1.0 (mặc định)
<code>reg_alpha</code> (α)	L1 regularization	0–10	0 (thử nếu sparse)
<code>gamma</code> (γ)	Min loss reduction to split	0–5	0 (tăng nếu overfit)
<code>min_child_weight</code>	Min sum hessian in leaf	1–10	1 (tăng nếu overfit)

Chiến lược: ① Fix $\eta=0.1$, tune `max_depth` + `subsample` ② Tìm `n_estimators` qua `early stopping` ③ Tune `reg_lambda`, `colsample` ④ Giảm η , tăng `n_estimators`

Cải tiến	Mô tả	Tác dụng
Column Block	Lưu data theo cột, sorted → cache-friendly	Tìm split nhanh hơn
Weighted Quantile Sketch	Approximate split tìm ngưỡng	Xử lý data lớn, distributed
Sparsity-Aware	Default direction cho missing values	Tự xử lý missing → không cần impute
Cache-Aware Access	Block size tối ưu cho CPU cache	2x nhanh hơn naive implementation
Out-of-Core	Chia data vào disk blocks	Dataset lớn hơn RAM
Regularization	L1 (α), L2 (λ), max_depth, γ	Giảm overfit mạnh hơn GB

Tiêu chí	GB truyền thống	XGBoost
Objective	Loss only	Loss + Regularization Ω
Optimization	Gradient (bậc 1)	Gradient + Hessian (bậc 2)
Missing values	Cần impute trước	Tự động xử lý
Parallelism	Không	Column-parallel split finding
Pruning	Pre-pruning (max_depth)	Post-pruning (Gain < 0)
Tốc độ	Chậm	Nhanh 10x+

06

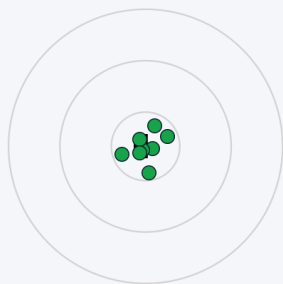
Bias–Variance & Thực Hành

"Lý thuyết nền tảng và so sánh thực nghiệm"

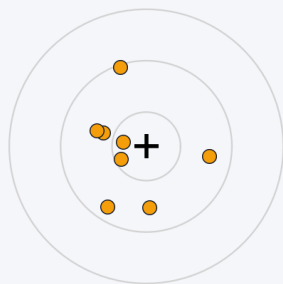


FPT SCHOOL OF BUSINESS
& TECHNOLOGY

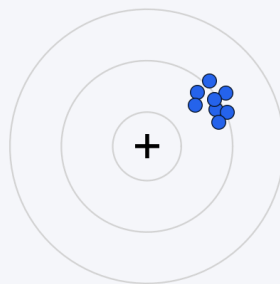
Low Bias
Low Variance



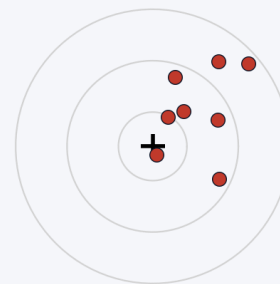
Low Bias
High Variance



High Bias
Low Variance



High Bias
High Variance

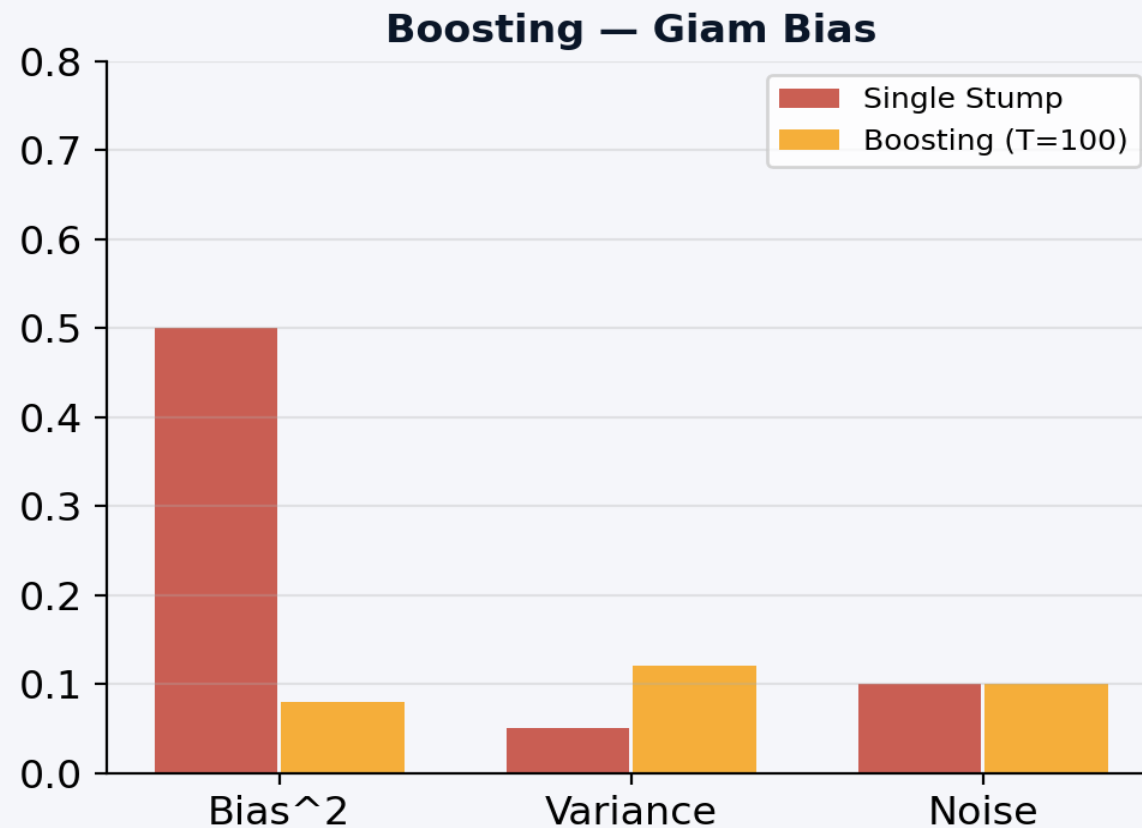
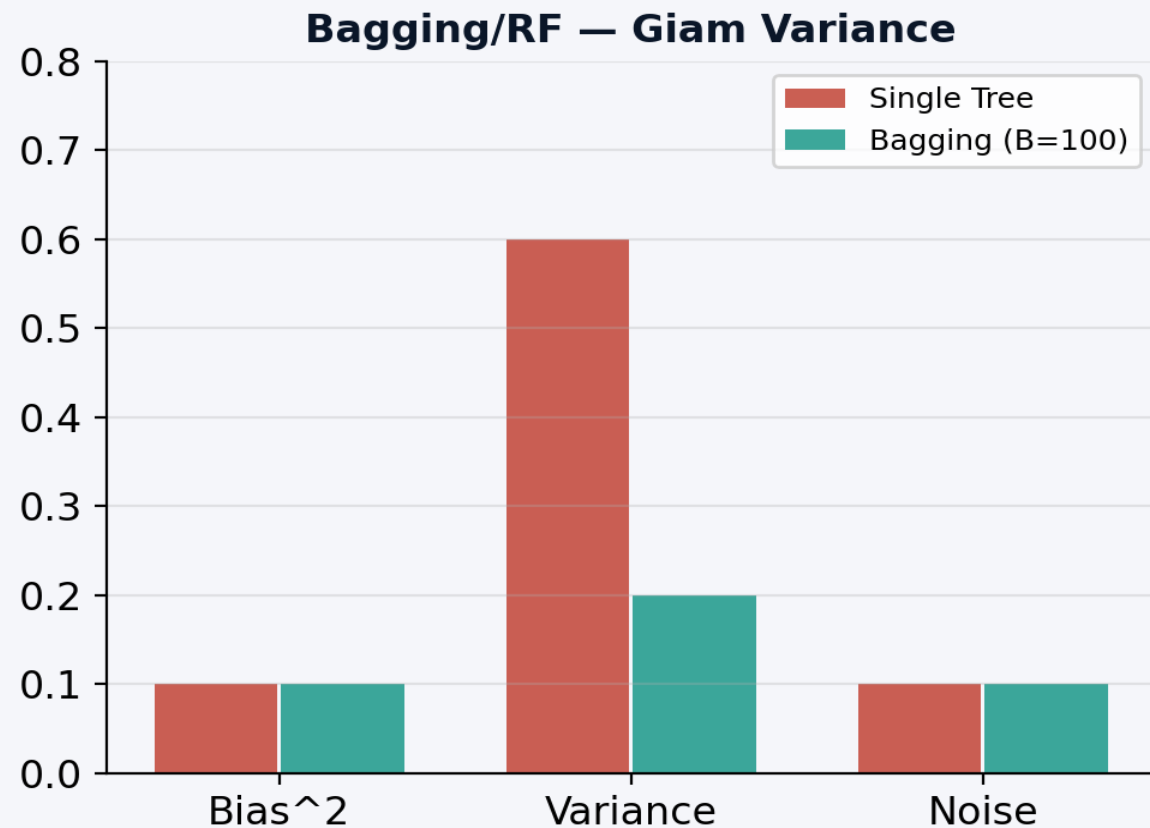


$$\text{Error}(x) = \text{Bias}^2 + \text{Variance} + \text{Noise}$$

Bias: sai lệch hệ thống
(model quá đơn giản)

Variance: dao động theo data
(model quá phức tạp)

	Bagging / RF	Boosting (AdaBoost, GB, XGB)
Base learner	Cây SÂU (low bias, high var)	Cây NÔNG / stumps (high bias, low var)
Chiến lược	Trung bình hóa → triệt tiêu variance	Sửa lỗi tuần tự → giảm bias
Bias sau ensemble	≈ Không đổi	Giảm mạnh
Variance sau ensemble	Giảm mạnh ($\div B$)	Có thể tăng (cần regularization)
Overfit risk	Thấp (tăng B an toàn)	Cao (cần early stopping, η nhỏ)



Bagging giảm VARIANCE (giữ bias thấp) | Boosting giảm BIAS (nhưng variance có thể tăng → cần η nhỏ + early stopping)

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import (BaggingClassifier, RandomForestClassifier,
                              AdaBoostClassifier, GradientBoostingClassifier)
import xgboost as xgb
from sklearn.model_selection import cross_val_score
models = {
    'Decision Tree': DecisionTreeClassifier(max_depth=5),
    'Bagging': BaggingClassifier(n_estimators=100),
    'Random Forest': RandomForestClassifier(n_estimators=200, max_features='sqrt'),
    'AdaBoost': AdaBoostClassifier(n_estimators=100, learning_rate=0.1),
    'Gradient Boost': GradientBoostingClassifier(n_estimators=200, learning_rate=0.1,
                                                max_depth=4, subsample=0.8),
    'XGBoost': xgb.XGBClassifier(n_estimators=200, learning_rate=0.1,
                                max_depth=5, subsample=0.8),
}
for name, model in models.items():
    scores = cross_val_score(model, X, y, cv=5, scoring='accuracy')
    print(f'{name:20s}: {scores.mean():.3f} ± {scores.std():.3f}')
```

Kết quả kỳ vọng: DT < Bagging < RF ≈ AdaBoost < GB < XGBoost (trên tabular data điển hình)

```
from sklearn.model_selection import GridSearchCV
import xgboost as xgb
# ===== Random Forest =====
rf_params = {
    'n_estimators': [100, 200, 500],
    'max_features': ['sqrt', 'log2', 0.3],
    'max_depth': [None, 10, 20],
}
rf_grid = GridSearchCV(RandomForestClassifier(), rf_params, cv=5, n_jobs=-1)
rf_grid.fit(X_train, y_train)
print(f'RF best: {rf_grid.best_score_:.3f} params: {rf_grid.best_params_}')
# ===== XGBoost + Early Stopping =====
xgb_params = {
    'max_depth': [3, 5, 7],
    'learning_rate': [0.01, 0.05, 0.1],
    'n_estimators': [200, 500],
    'subsample': [0.7, 0.8, 1.0],
    'colsample_bytree': [0.7, 0.8, 1.0],
    'reg_lambda': [0.1, 1.0, 5.0],
}
xgb_grid = GridSearchCV(xgb.XGBClassifier(early_stopping_rounds=20,
                                         eval_metric='logloss'),
                        xgb_params, cv=5, n_jobs=-1)
xgb_grid.fit(X_train, y_train,
            eval_set=[(X_val, y_val)], verbose=False)
print(f'XGB best: {xgb_grid.best_score_:.3f} params: {xgb_grid.best_params_}')
```

07

So sánh & Tổng Kết

"Bức tranh toàn cảnh Ensemble Methods"



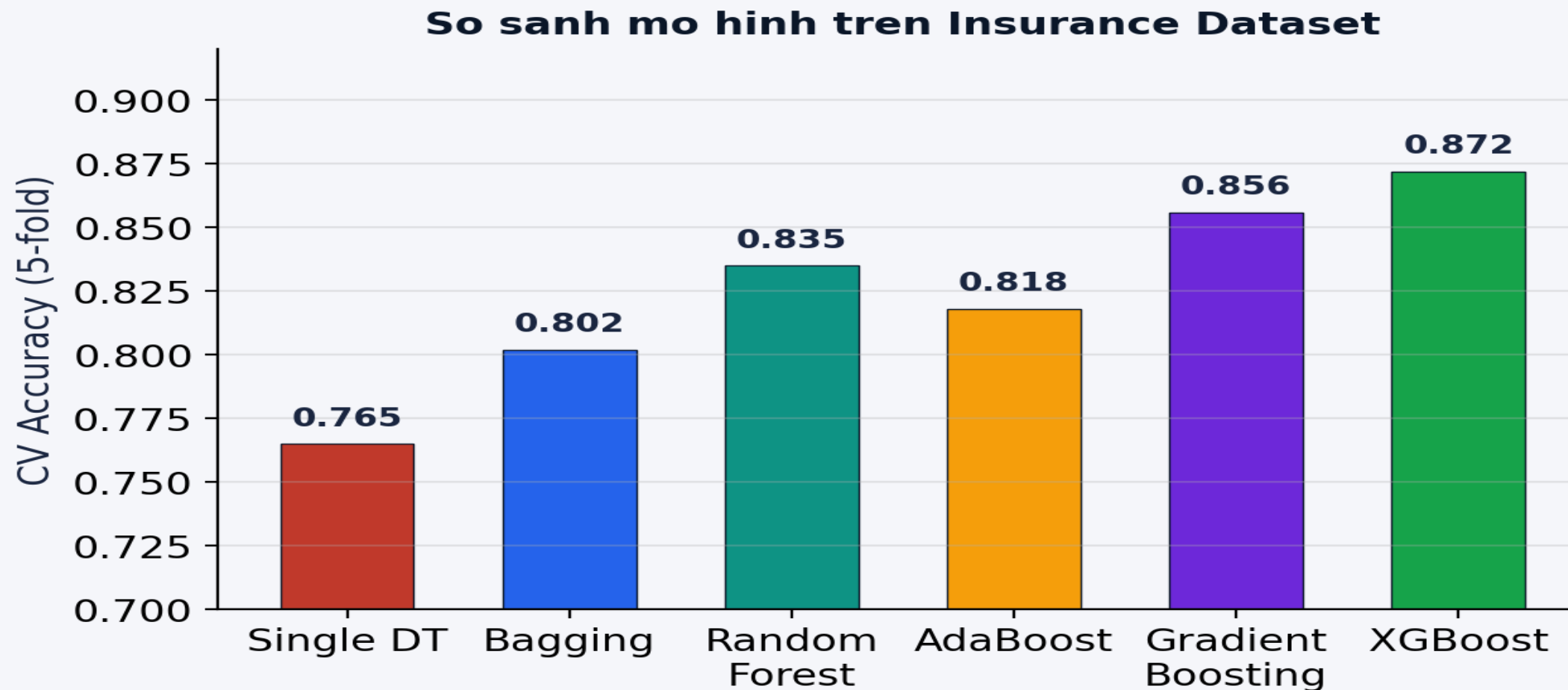
FPT SCHOOL OF BUSINESS
& TECHNOLOGY

Bảng so sánh toàn diện

30

	Bagging	Random Forest	AdaBoost	Gradient Boost	XGBoost
Kiểu	Parallel	Parallel	Sequential	Sequential	Sequential
Giảm	Variance	Variance	Bias	Bias	Bias
Base learner	Cây sâu	Cây sâu	Stump	Cây nông	Cây nông
Regularization	Không	Feature subset	α weights	η , subsample	$\eta, \lambda, \gamma, \alpha$
Missing values	Impute	Impute	Impute	Impute	Tự động
Overfit risk	Thấp	Thấp	Trung bình	Cao	Trung bình
Tốc độ train	Nhanh (parallel)	Nhanh	Trung bình	Chậm	Nhanh
Accuracy (typical)	Trung bình	Cao	Cao	Rất cao	Rất cao





Decision Flowchart — Chọn Ensemble nào?

32

Cần baseline nhanh,
ít tuning?

→ Random Forest

Cần accuracy tối đa
trên tabular data?

→ XGBoost / LightGBM

Data ít noise,
muốn đơn giản?

→ AdaBoost

Cần interpretability?

→ Single DT hoặc
RF + Feature Importance

Data có missing values,
cần nhanh?

→ XGBoost (native)

Data rất lớn
(>1M rows)?

→ LightGBM (GOSS+EFB)

Thực tế: Thử RF baseline → nếu cần hơn → XGBoost/LightGBM + GridSearchCV. Luôn bắt đầu đơn giản!



Tổng kết — Key Takeaways

33

01 Bootstrap & Bagging

Lấy mẫu có hoàn lại $\rightarrow$ train B cây
song song $\rightarrow$ trung bình $\rightarrow$ giảm
variance

02 Random Forest

Bagging + Feature Randomness
 $\rightarrow$ giảm $\rho \rightarrow$ variance giảm mạnh
hơn. Go-to baseline!

03 AdaBoost

Reweight samples sai $\rightarrow$ weak
learners $\rightarrow$ strong learner. Nhạy
noise.

04 Gradient Boosting

Fit residuals (pseudo-residuals) $\rightarrow$
ANY loss function. η + early
stopping.

05 XGBoost

GB + Regularization + Newton
bậc 2 + Engineering $\rightarrow$ nhanh,
mạnh, Kaggle winner.

06 Bias–Variance

Bagging giảm Variance, Boosting
giảm Bias. Hiểu để chọn đúng
phương pháp.

